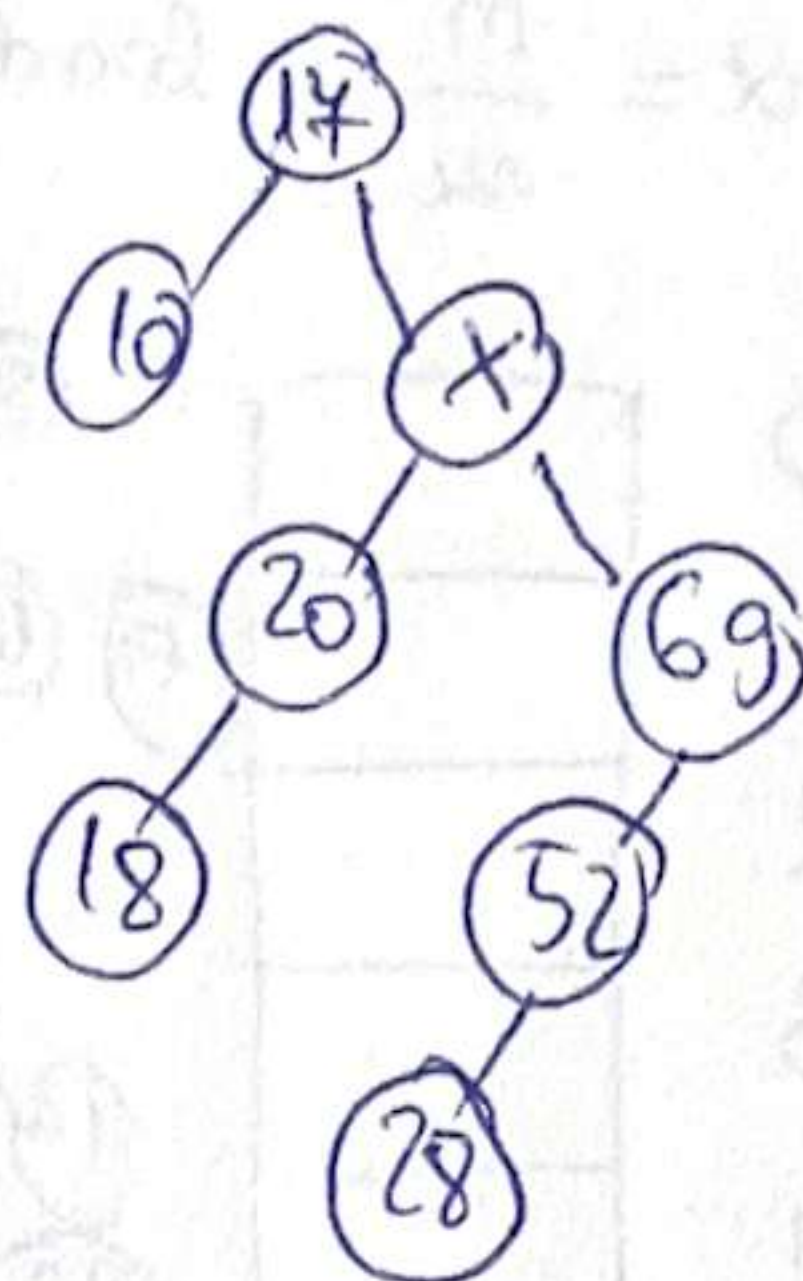


DSA mock

A.

1) BST ; $x = ?$

e) 26 ; f) 23

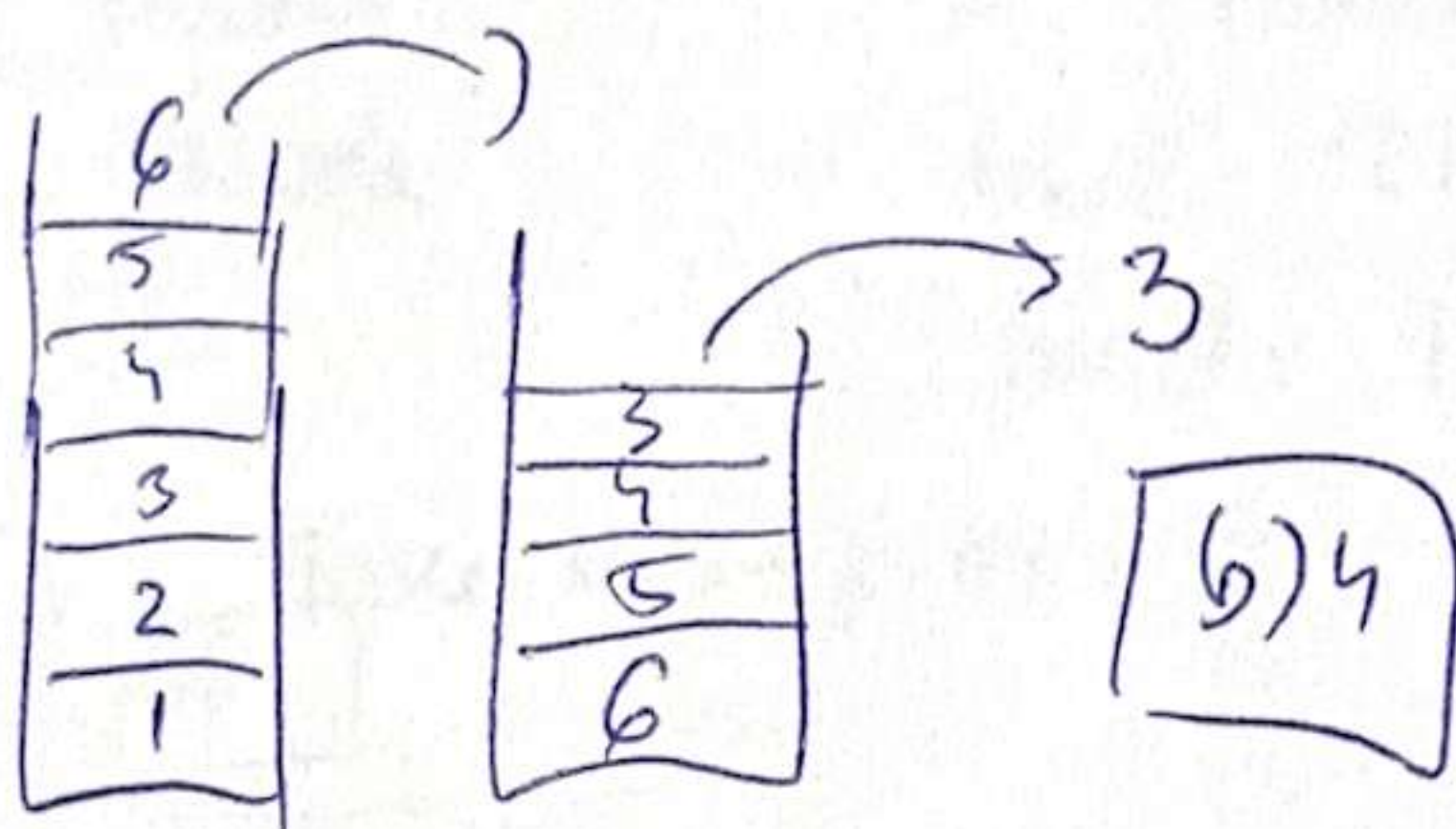


2) a) $\Omega(m^3)$

b)

c)

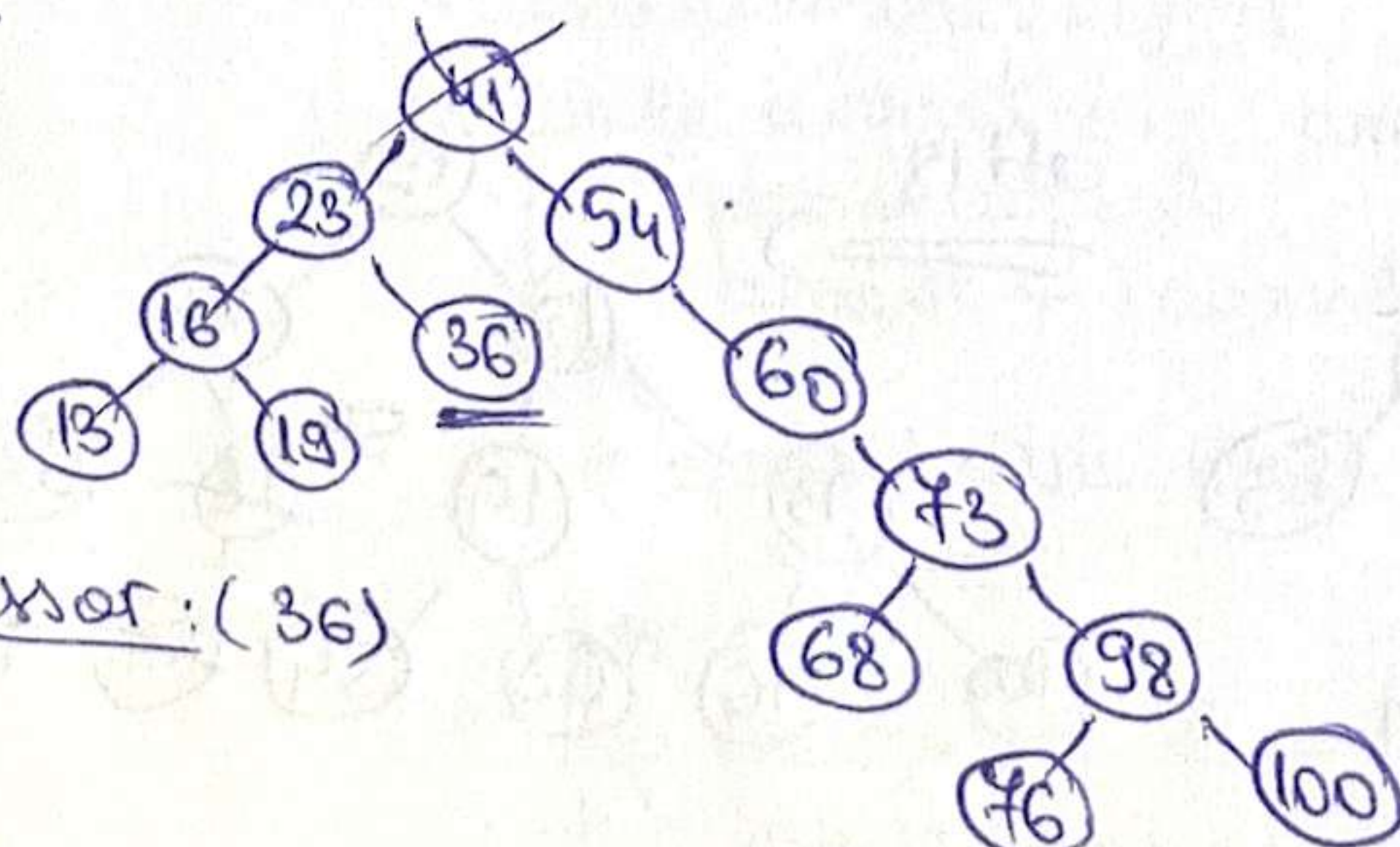
3)



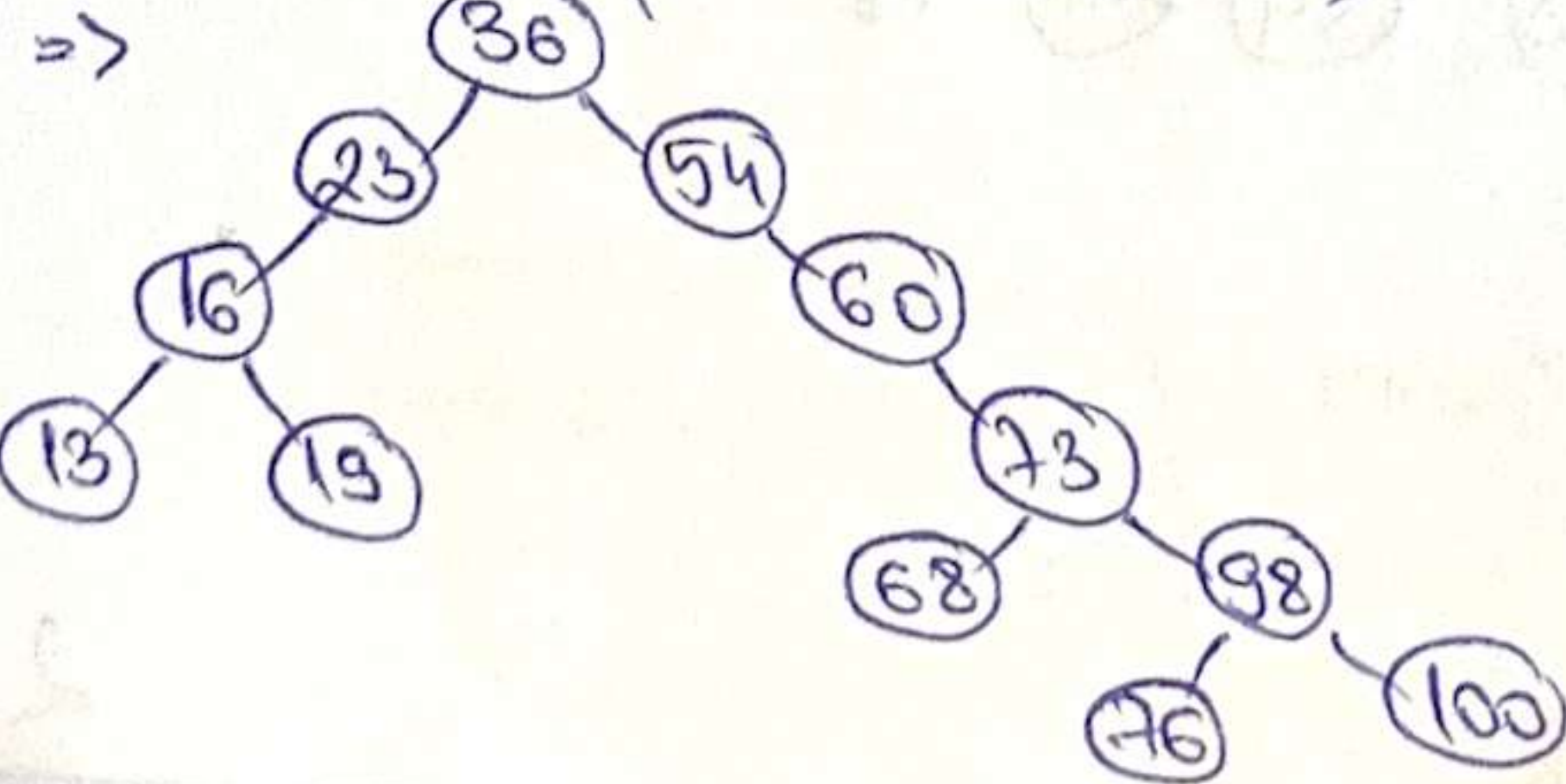
4)

Token	Stack
4	4
6	4 6 ←
7	4 6 7 ←
+	4 13
1	4 13 1
-	4 12
2	4 12 2
5	4 12 2 5
*	4 12 16
+	4 24
+	28

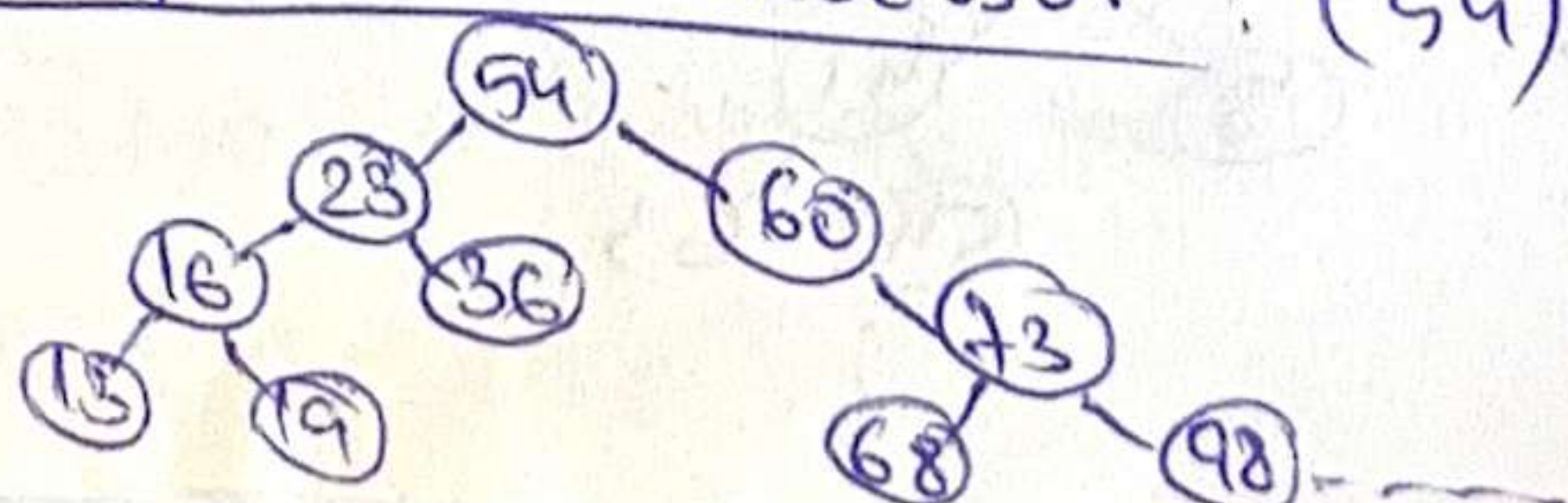
B. 2.1.) BST.



I. replace w predecessor : (36)



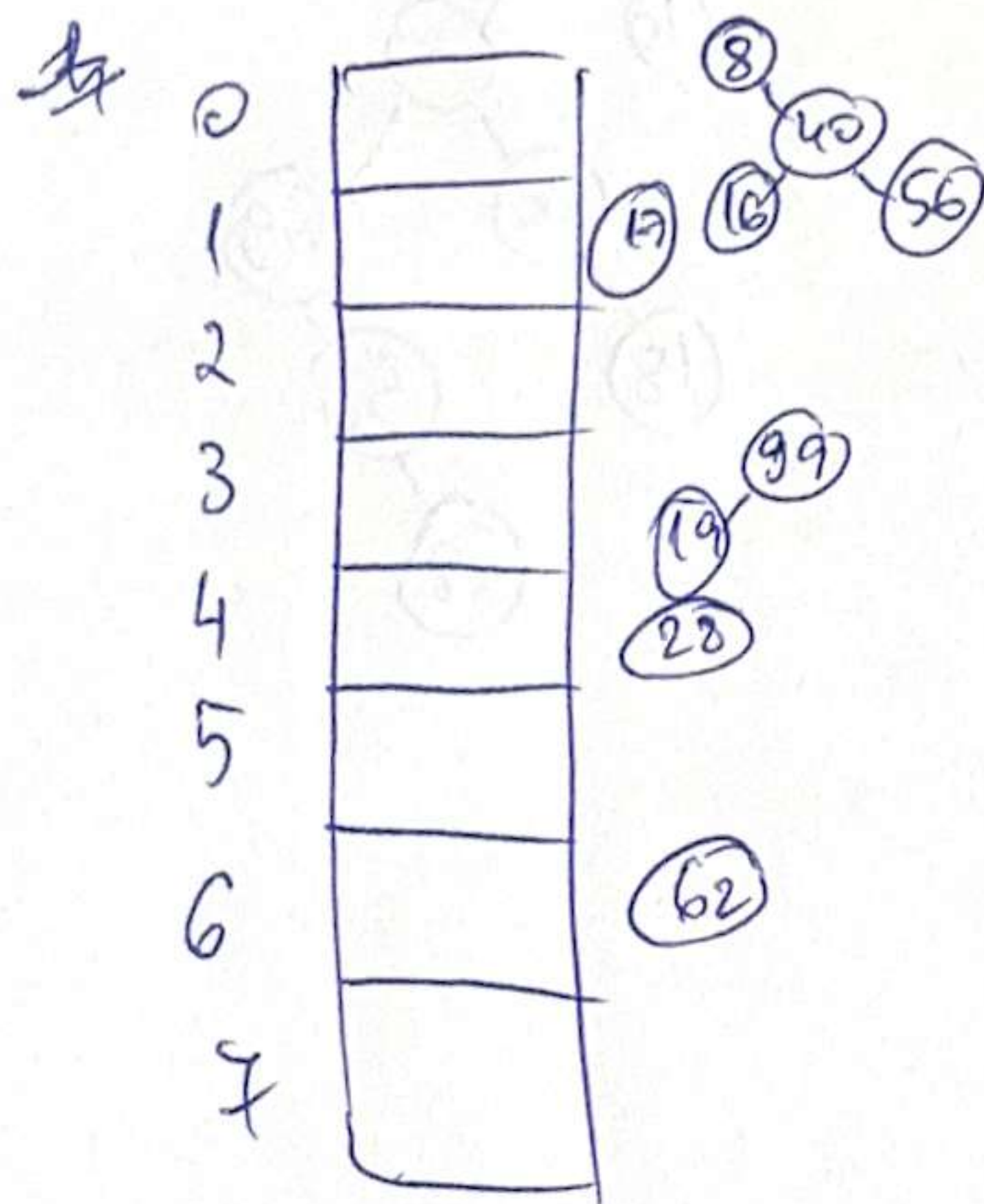
II. replace w successor : (54)



2.2) HT, $m=8$, collision res. w sep. chaining

add: 8, 99, 40, 19, 56, 16, 17, 28, 62. $\Rightarrow m=9$

$$\alpha = \frac{m}{n} = \text{load factor} = \frac{9}{8}$$



$$h(8) = 8 \% 8 = 0 \checkmark$$

$$h(99) = 3 \checkmark$$

$$h(40) = 0 \checkmark$$

$$h(19) = 3 \checkmark$$

$$h(56) = 0 \checkmark$$

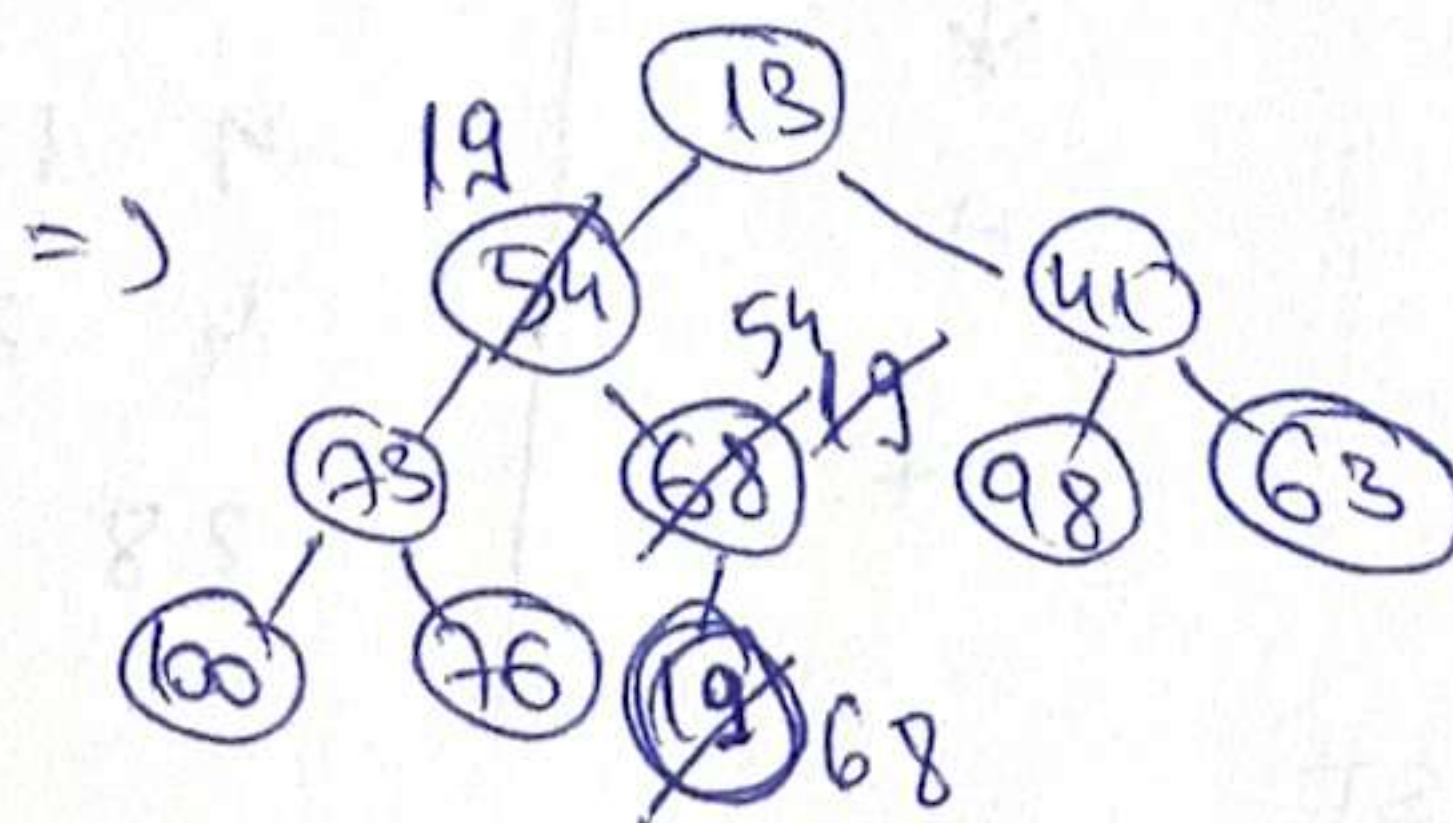
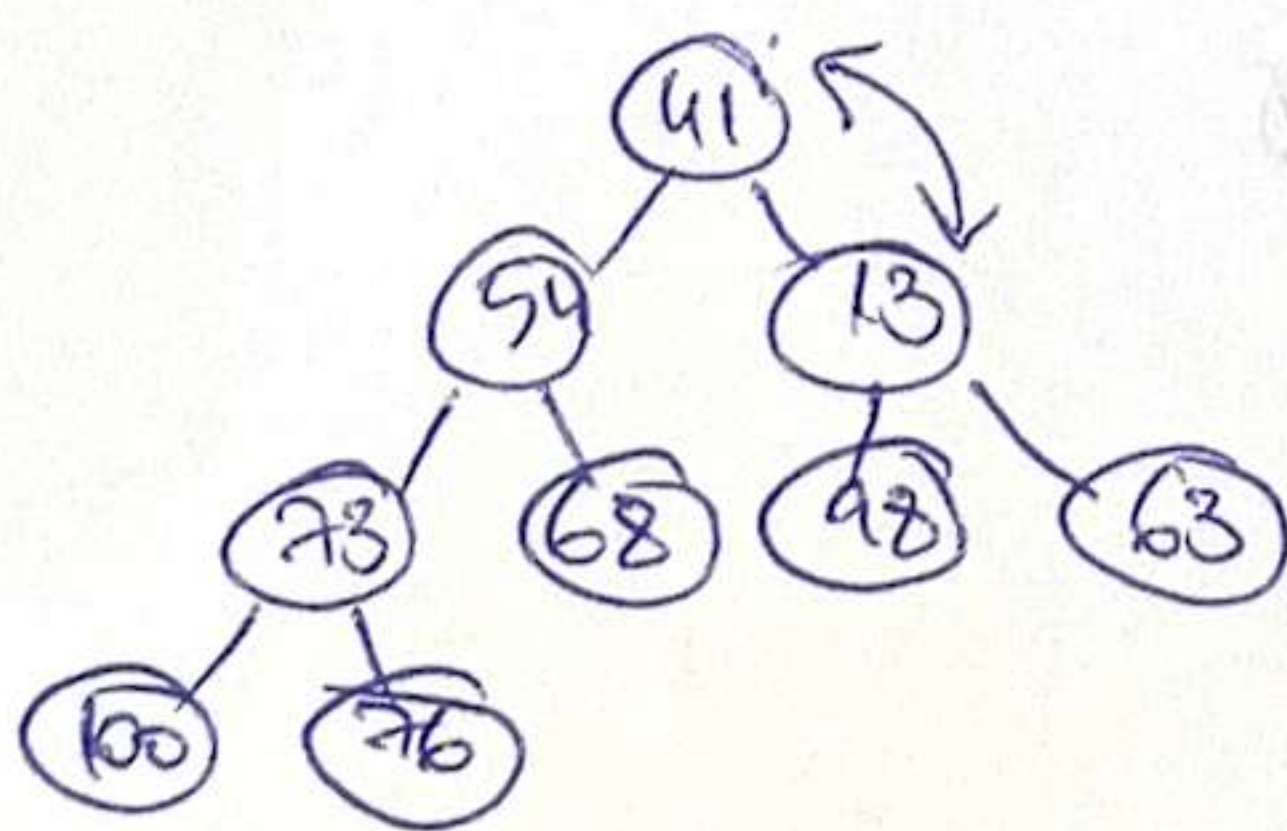
$$h(16) = 0 \checkmark$$

$$h(17) = 1 \checkmark$$

$$h(28) = 4 \checkmark$$

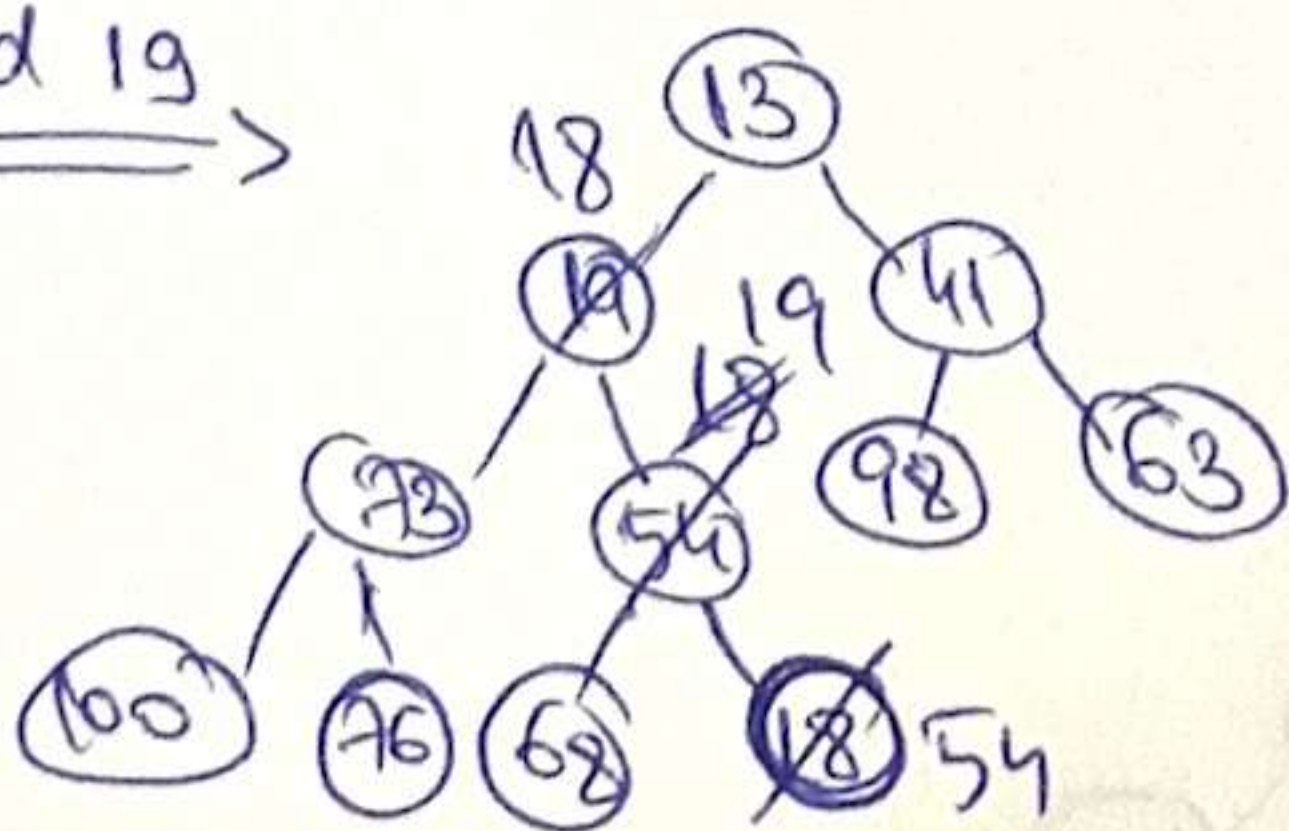
$$h(62) = 6$$

2.3)

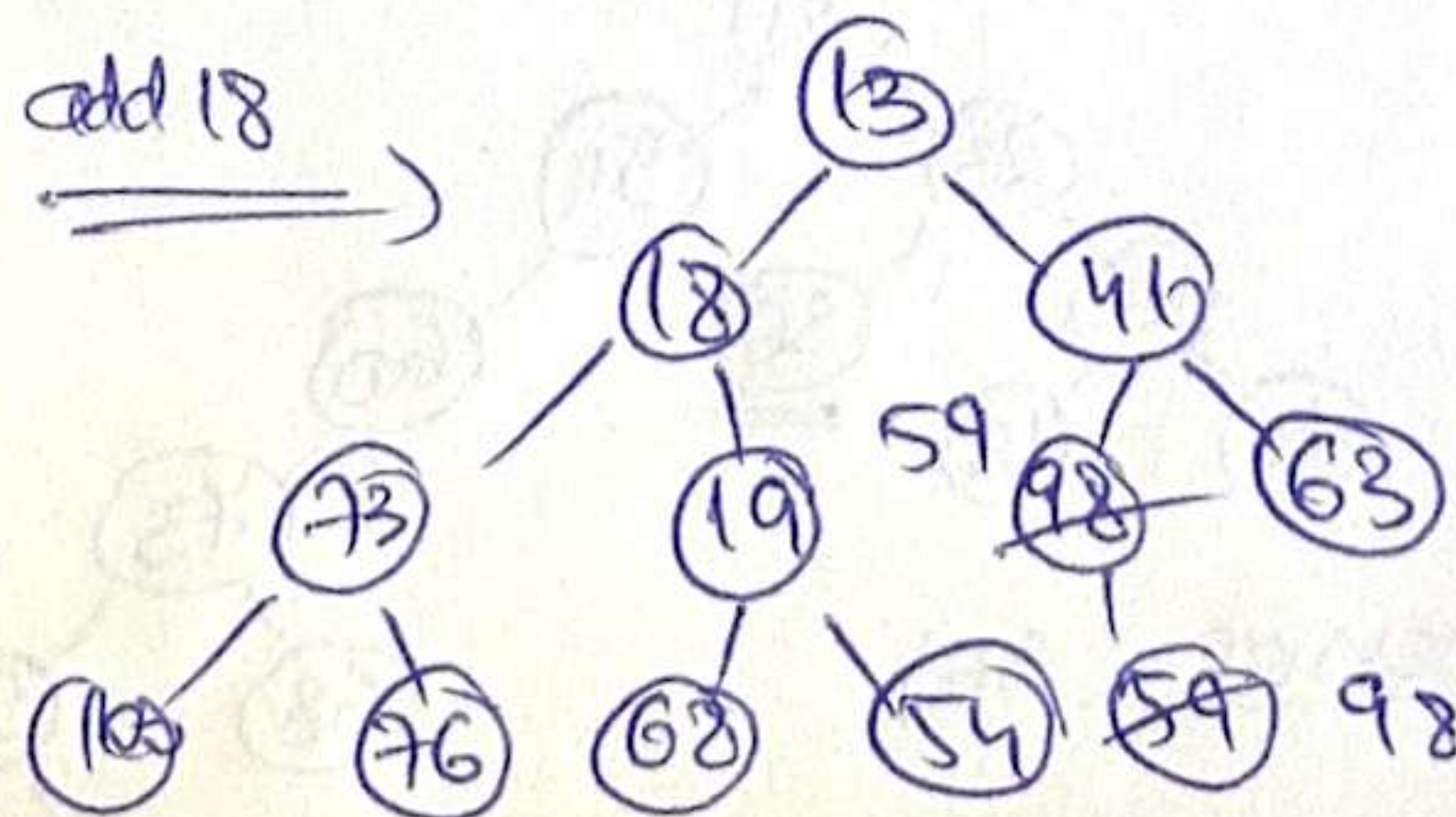


min-heap

add 19

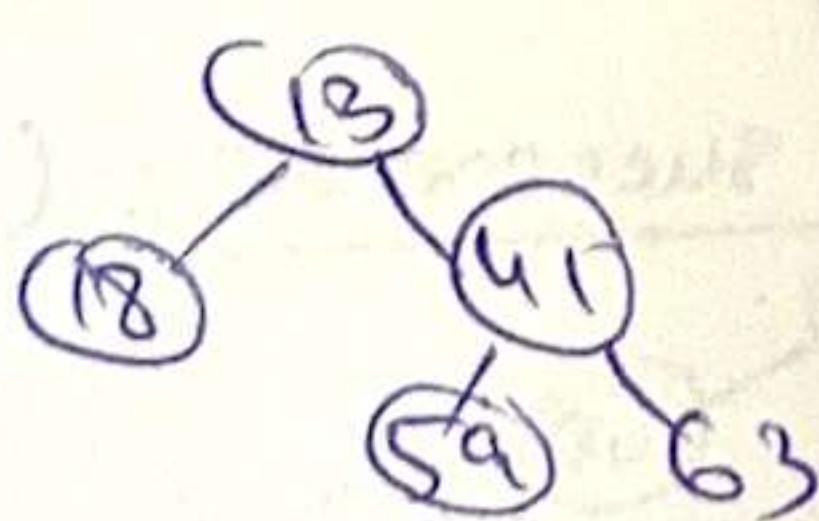


add 18



add 59

$\Rightarrow$



$\Rightarrow$ remove ...

DS: - search, insert, remove: same complexity as a HT.
 - iteration order = insertion order $\rightarrow \Theta(1)$ iterators.

Search $\Theta(1)$
 insert $\Theta(1)$
 remove $\Theta(1)$

a) Repres. = Linked Hash Table

Node:

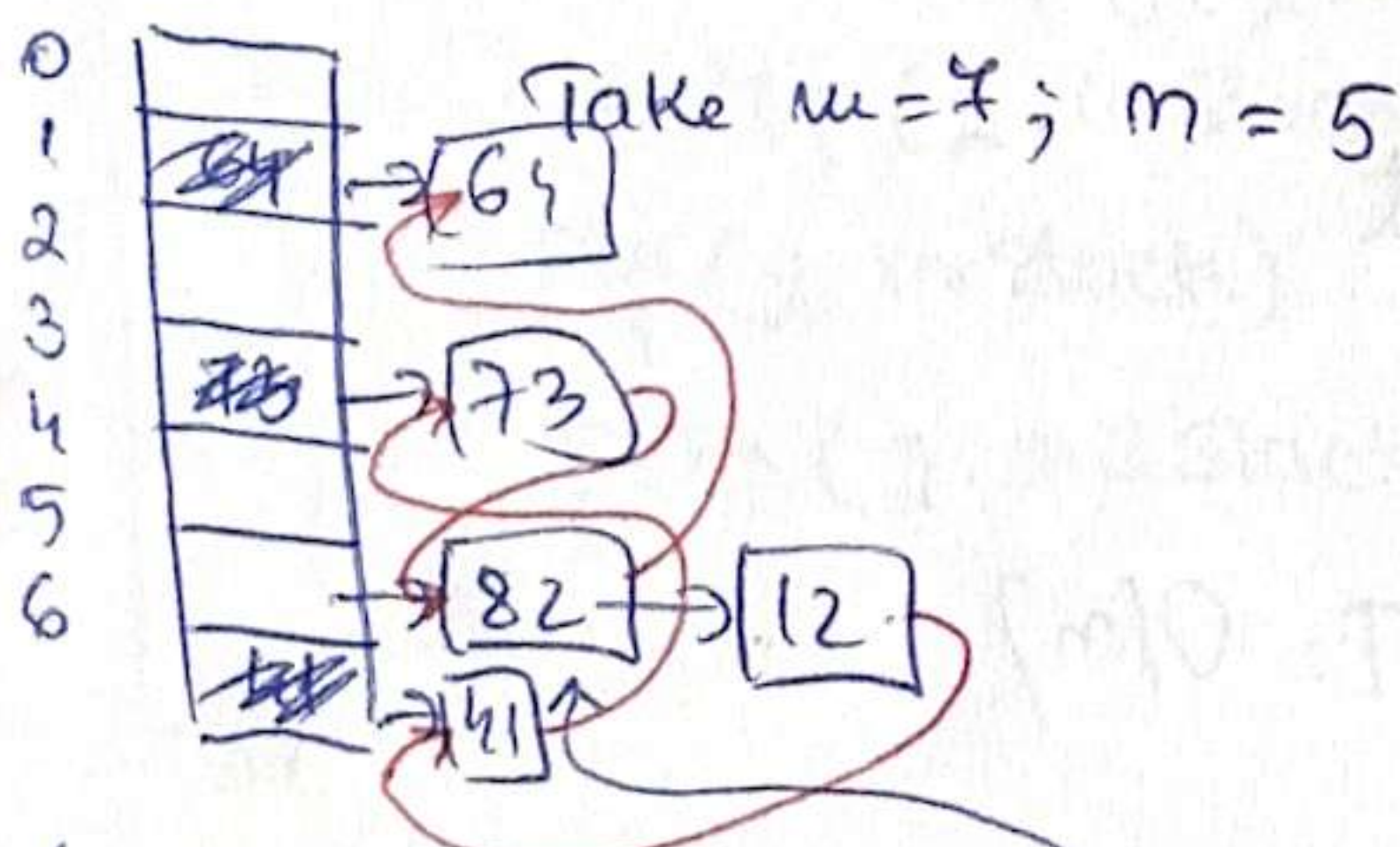
info: TKey
 nextH: $\uparrow$ Node
 nextL: $\uparrow$ Node
 prevL: $\uparrow$ Node

Linked HT

m: Integer
 T: (TNode)[]
 h: TFunction
 tail: $\uparrow$ Node
 head: $\uparrow$ Node

gives
 insertion-
 order iteration
 with $\Theta(1)$ op.

b)



$$h(12) = 12 \% 7 = 5$$

$$h(41) = 41 \% 7 = 6$$

$$h(73) = 3$$

$$h(82) = 5 \text{ insert at the beginning}$$

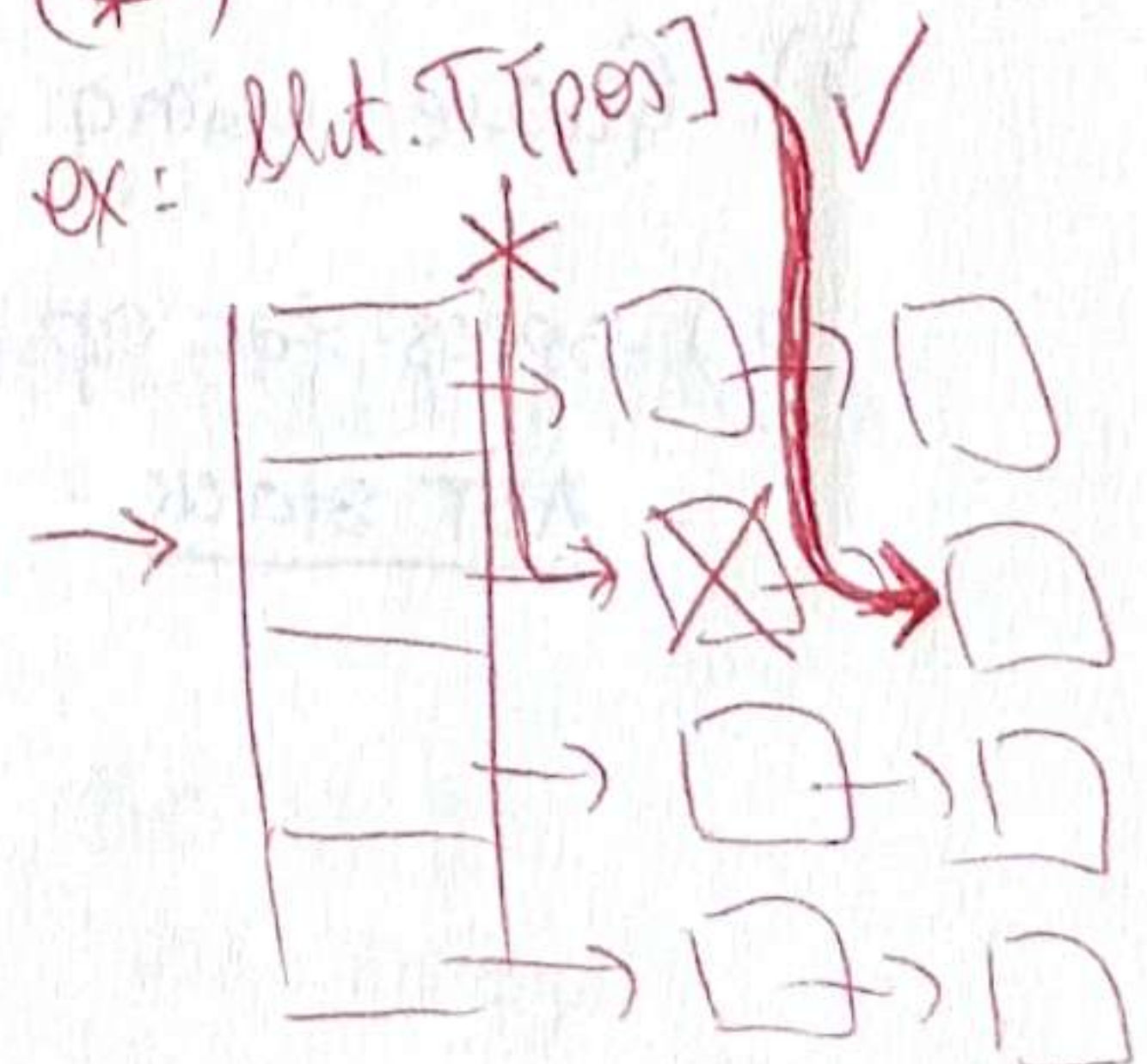
$$h(64) = 1$$

$\Rightarrow 12 \leftrightarrow 41 \leftrightarrow 73 \leftrightarrow 82 \leftrightarrow 64$

$\uparrow$
 head

$\uparrow$
 tail

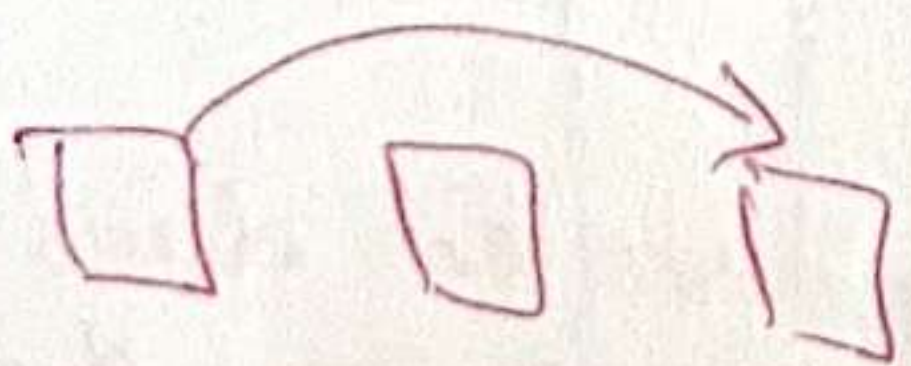
(*)



c) remove

~~function remove~~

(**):



subalgorithm remove (llt, e) is

$pos \leftarrow llt.h(e)$

$prev \leftarrow Nil$

$current \leftarrow llt.T[pos]$

while $current \neq Nil$ and $[current].info \neq e$ ex

$prev \leftarrow current$

$current \leftarrow [current].nextH$

if $current = Nil$ then

return

(*) if $prev = Nil$ then // we have to delete the first node of the LL on that pos.

$llt.T[pos] \leftarrow [current].nextH$

(**) else

$[prev].nextH \leftarrow [current].nextH$

remove
 from hash
 collision chain

if $[current].prevL = Nil$ then // first pos in the LL.
 $lht.head \leftarrow [current].nextL$

else
 $[[current].prevL].nextL \leftarrow [current].nextL$

if $[current].nextL = Nil$ then // last pos in the LL
 $lht.tail \leftarrow [current].prevL$

else
 $[[current].nextL].prevL \leftarrow [current].prevL$

deallocate current

Bc: $\Theta(1)$

Wc: $\Theta(m)$

Ac: $\Theta(1)$

$\Rightarrow T: \Theta(m)$

4) Queue using 2 stacks.

a) specs for op of the stack

ADT Stack:

- init(s)
- push(s, e)
- pop(s)
- isEmpty(s)
- size(s)

$\} \Theta(1)$

Stack:

capacity: Int

nrElem: Int

elems: TElem[]

b) ADT Queue

Queue:

inStack: Stack $\rightarrow$ stores newly inserted elems

outStack: Stack $\rightarrow$ stores elems ready to leave the queue

c) specify + implement all Queue op:

- init(q)

- push(q, e)

- pop(q)

- isEmpty(q)

- front(q)

d) compl.

subalgorithm $init(q)$ is

1. $inStack \leftarrow init(inStack)$
2. $outStack \leftarrow init(outStack)$

$\Theta(1)$

subalgorithm $push(q, e)$ is $\Theta(1)$

$push(q.inStack, e)$

subalgorithm $pop(q, e)$ is

if $isEmpty(q.outStack)$ then

while not $isEmpty(q.inStack)$ execute

$x \leftarrow top(q.inStack)$

$pop(q.inStack)$

$push(q.outStack, x)$

$pop(q.outStack)$

BC: $\Theta(1)$

WC: $\Theta(n)$

function $isEmpty(q)$ is $\Theta(1)$

$isEmpty \leftarrow isEmpty(q.inStack) \text{ and } isEmpty(q.outStack)$

function $front(q)$ is

if $isEmpty(q.outStack)$ then

while not $isEmpty(q.inStack)$ ex

$x \leftarrow top(q.inStack)$

$pop(q.inStack)$

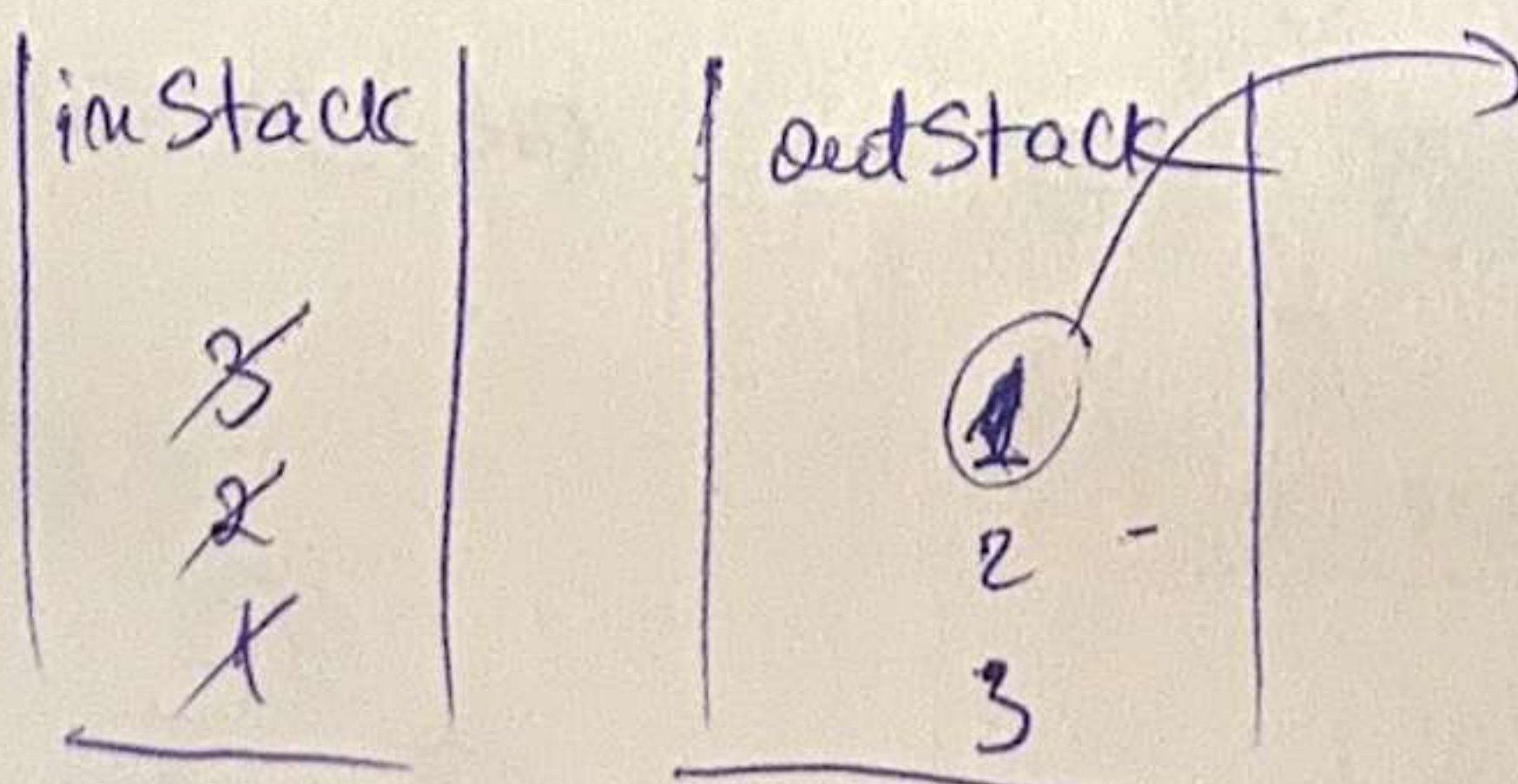
$push(q.outStack, x)$

$front \leftarrow top(q.outStack)$

BC: $\Theta(1)$

WC: $\Theta(n)$

e)



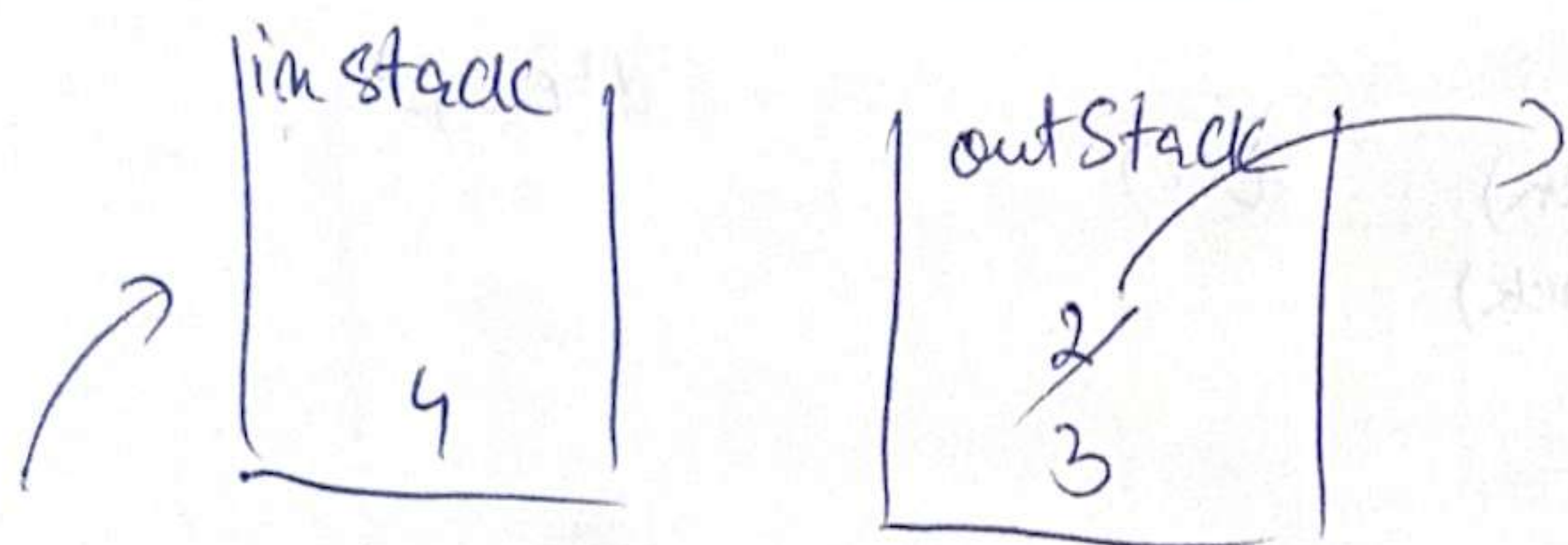
queue: 1, 2, 3 $\Rightarrow$ queue: 2, 3

push 1, 2, 3 to q .

pop 1 elem.

add elem 4

pop 1 elem



2) queue: 2, 3, 4

2) queue: 3, 4

Queue order = outStack + inStack

